

LION BIOLOGGING & BEHAVIORAL EXTRACTION VIA DEEP AUTOENCODERS

SoS'26 Mid-term Report

Project Author:

Kartik Bunas

Roll No:

25B0912

Mentor:

Pankti Desai

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING,
INDIAN INSTITUTE OF TECHNOLOGY BOMBAY**

Table of Contents

1	Project Overview & Objectives	2
2	Data Preprocessing Pipeline	2
2.1	3D Sequence Formulation	2
2.2	Handling Sensor Anomalies (NaN Dropout)	2
2.3	Flat Multi-Dimensional Scaling	3
3	LSTM Autoencoder Design	3
3.1	Input Gate & Encoder Pipeline	4
3.2	Dimension Reconstruction Bridge & Decoder Pipeline	4
3.3	Output Gate Allocation	4
4	Model Training & Convergence Analysis	5
5	Updated Plan of Action (Weeks 5 – 7)	5

1 Project Overview & Objectives

This project focuses on the unsupervised classification of lion behavior using high-frequency bio-logging telemetry data. Field deployment of movement sensors yields vast streams of continuous data, capturing metrics like speed, acceleration, and heading changes. Labeling this data manually through direct observation is logistically unfeasible. Therefore, the goal of this project is to build an end-to-end machine learning pipeline that ingests raw telemetry sequences, filters out sensor-induced noise, and learns a low-dimensional representation space where distinct behavioral states (e.g., resting, patrolling, or hunting) can be separated.

During the first four weeks, the focus has been on constructing the core data engineering pipeline and building the neural network architecture. An LSTM-based Autoencoder was designed and successfully trained on the laptop’s local hardware (Ryzen 5 CPU and GTX 1650 GPU). By forcing the network to compress 24-hour windows of data down to a small bottleneck layer before reconstructing the original sequence, the network learns to isolate the underlying behavioral signals from stochastic sensor noise.

2 Data Preprocessing Pipeline

2.1 3D Sequence Formulation

The raw telemetry data consists of a continuous time-series. To analyze behavioral patterns on a daily cycle, the continuous data was chunked into discrete 24-hour windows. Each window contains 24 hourly steps, and each step tracks 8 physical features (including step length, velocity, turning angles, and tri-axial acceleration signals). This creates a 3D matrix array of shape:

$$X \in \mathbb{R}^{N \times T \times F} \quad (1)$$

where the initial matrix loading yielded $N = 15,953$ sequences, $T = 24$ timesteps, and $F = 8$ features.

2.2 Handling Sensor Anomalies (NaN Dropout)

During initial passes, data scaling operations failed because of hidden NaN values in the tracking logs, caused by canopy signal blockages or momentary device reboots. To clean the matrix without altering the sequence timelines, a multi-dimensional Boolean reduction mask was applied to scan the 3D tensor across the temporal and feature axes simultaneously:

$$M_i = \neg \left(\bigvee_{j=1}^{24} \bigvee_{k=1}^8 \text{is_nan}(X_{i,j,k}) \right) \quad (2)$$

The cleanup was executed using the following implementation:

```

1 # Find which 24-hour windows are 100% clean (no NaNs across hours or
  features)
2 clean_mask = ~np.isnan(X_3D).any(axis=(1, 2))
3
4 # Keep only the rows that pass the mask
5 X_3D_clean = X_3D[clean_mask]

```

The mask isolated exactly one corrupted 24-hour sequence. Dropping this window reduced the dataset to 15,952 completely pristine daily sequences, establishing data stability for gradient calculations.

2.3 Flat Multi-Dimensional Scaling

The 8 tracking features operate on vastly different raw numerical bounds. To prevent features with large absolute ranges from dominating the network’s weight updates, the data required normalization to a uniform $[0, 1]$ range.

Because standard scaling tools operate on 2D arrays, the clean 3D array was flattened into a temporary 2D matrix of shape $(15952 \times 24, 8) = (382848, 8)$. MinMax scaling was applied to each feature column independently before reshaping back to the native 3D layout:

```

1 nsamples, ntimesteps, nfeatures = X_3D_clean.shape
2 X_flat = X_3D_clean.reshape(-1, nfeatures)
3
4 scaler = MinMaxScaler(feature_range=(0,1))
5 X_flat_scaled = scaler.fit_transform(X_flat)
6 X_scaled = X_flat_scaled.reshape(nsamples, ntimesteps, nfeatures)

```

Post-transformation assertions confirmed global minimum and maximum values of exactly 0.0 and 1.0.

Table 1: Data Pipeline Phase Progress

Pipeline Phase	Samples (N)	Hours (T)	Features (F)	Data Status
Raw Loading	15953	24	8	Contains unmapped NaN
Boolean Mask Execution	15952	24	8	1 corrupt row removed
2D Flattening Array	382848	–	8	Scale variance unadjusted
MinMax Scaled Array	15952	24	8	Homogeneous range $[0.0, 1.0]$

3 LSTM Autoencoder Design

The model was built using the Keras Functional API. The architecture consists of an Encoder that compresses data sequences and a Decoder that attempts to reconstruct them.

3.1 Input Gate & Encoder Pipeline

The input layer initializes a tensor allocation footprint of shape $(None, 24, 8)$ to accommodate variable-sized training mini-batches. This feeds directly into an LSTM Encoder layer configured with 16 computational hidden units.

By setting `return_sequences=False`, the encoder is constrained to output only its final hidden state vector after processing the 24th hour. This effectively acts as the information bottleneck, compressing the 192 raw tracking components (24×8) into a flat 16-dimensional latent vector representing the core behavioral summary of that day.

```
1 from tensorflow.keras.layers import Input, LSTM
2
3 inputs = Input(shape=(ntimesteps, nfeatures), name="Input_Gate")
4 encoder = LSTM(units=16, return_sequences=False, name="LSTM_Encoder")(
    inputs)
```

3.2 Dimension Reconstruction Bridge & Decoder Pipeline

The output of the encoder is a flat 2D tensor of shape $(None, 16)$. Because the downstream recurrent layers require a sequential time-axis to compute reconstructions, a `RepeatVector` layer is used to copy the 16-dimensional representation exactly 24 times. This creates a $(None, 24, 16)$ workspace tensor.

The Decoder LSTM layer reads this sequence and processes it. By setting `return_sequences=True`, the Decoder outputs its internal hidden states for each individual hour slot, yielding an output tensor shape of $(None, 24, 16)$.

```
1 from tensorflow.keras.layers import RepeatVector
2
3 bridge = RepeatVector(n=ntimesteps, name="Bridge_RepeatVector")(encoder
    )
4 decoder_lstm = LSTM(units=16, return_sequences=True, name="LSTM_Decoder
    ")(bridge)
```

3.3 Output Gate Allocation

The final output layer maps the 16 internal processing tracks back to the original 8 feature values. To do this across all steps without losing the time dimension, a linear `Dense(8)` layer is embedded inside a `TimeDistributed` wrapper. This ensures that the 16-to-8 feature contraction is executed independently for each of the 24 individual hours, restoring the native sequence template.

```
1 from tensorflow.keras.layers import TimeDistributed, Dense
2 from tensorflow.keras.models import Model
3
4 outputs = TimeDistributed(Dense(units=nfeatures), name="Output_Gate")(
    decoder_lstm)
5 lion_autoencoder = Model(inputs=inputs, outputs=outputs, name="
    Lion_Autoencoder_Model")
```

4 Model Training & Convergence Analysis

The network was compiled using the Adam optimizer with its default adaptive learning parameters. The optimization criterion was Mean Squared Error (`loss='mse'`), calculating the squared difference between the true normalized values (X) and the model's reconstructions ($\hat{X}$):

$$\text{MSE} = \frac{1}{24 \times 8} \sum_{j=1}^{24} \sum_{k=1}^8 \left(X_{i,j,k} - \hat{X}_{i,j,k} \right)^2 \quad (3)$$

Training was executed with a batch size of 32, shuffling data samples each epoch. Ten percent of the data (1,595 day sequences) was split off into a validation set to monitor generalization performance. Over 20 training epochs, the reconstruction loss dropped steadily. The loss values for the final epochs are documented in Table 2.

Table 2: Loss Convergence Tracking

Epoch	Total Batches	Step Speed	Training Loss (MSE)	Validation Loss (MSE)
Epoch 15	449 / 449	20 ms/step	0.0581	0.0593
Epoch 16	449 / 449	24 ms/step	0.0577	0.0590
Epoch 17	449 / 449	24 ms/step	0.0574	0.0588
Epoch 18	449 / 449	20 ms/step	0.0573	0.0586
Epoch 19	449 / 449	24 ms/step	0.0571	0.0585
Epoch 20	449 / 449	20 ms/step	0.0570	0.0585

The training loss converged smoothly to 0.0570, and the validation loss tracked it closely at 0.0585. The minimal gap between training and validation performance confirms that the network has generalized its reconstruction rules effectively across sequences without memorizing random sensor variations or overfitting. Both the trained weights matrix (`.keras`) and preprocessed arrays (`.npy`) have been saved to storage.

5 Updated Plan of Action (Weeks 5 – 7)

With data preparation, preprocessing pipelines, and autoencoder training complete, the updated timeline for the remainder of the project is structured as follows:

- **Week 5: Latent Space Extraction & Dimensionality Verification**

Isolate the trained Encoder block from the complete autoencoder. Pass all 15,952 normalized tracking rows through it to extract the corresponding 16-dimensional latent feature summaries. Apply PCA and t-SNE projections to map and visually evaluate the structural grouping density of this low-dimensional representation space.

- **Week 6: Unsupervised Clustering & Behavior Isolation**

Implement K-Means and Gaussian Mixture Models (GMM) on the extracted 16-dimensional behavioral signatures. Use silhouette scores and the Elbow method to determine the optimal number of clusters (K), partitioning the latent representations into distinct movement archetypes.

- **Week 7: Coordinate Reconstruction & Final Project Synthesis**

Map the discovered clusters back onto geographic GPS trajectories to verify behavioral classifications against real-world biological movements. Finalize compilation of the full-term project report and prepare the code repositories for submission.

By:
Kartik Bunas
25B0912